LA Transformación CONTINÚA

INSTITUCIÓN UNIVERSITARIA
PASCUAL BRAVO®

Informe técnico comparativo de tres SGBD

Experto temático:

Juan Camilo Villegas Ocampo

Docente:

Oralia Cortez Grajales

Medellín 25 / 10 / 2026

Unidad de Educación Digital

VIGILADA Mineducación

Alcaldía de Medellín

Contenido

1. Introducción
2. Justificación de la elección de los tres SGBD
3. Diccionario de datos genérico actualizado
4. Implementación paso a paso en cada SGBD
5. Comparación de los diccionarios físicos
6. Tipos de datos SQL y NoSQL para big data e IoT
7. Retroalimentación y ajustes
8. Dificultades, ventajas y desventajas por SGBD
9. Conclusiones y propuestas de mejora
10. Referencias
11. Anexo: estructura del repositorio

1. Introducción

Este informe compara la implementación del diccionario de datos físico de la Red Social Estudiantil Pascualina en tres sistemas gestores de bases de datos: PostgreSQL, MySQL y Microsoft SQL Server. Parte del modelo lógico normalizado (3FN) de la Tarea 2, que se implementó en PostgreSQL en la Tarea 3, y lo adapta a los otros dos motores, teniendo en cuenta que cada uno tiene sus propios tipos de datos, sintaxis y limitaciones.

Además de traducir el modelo, el reto incluyó configurar la base de datos para manejar datos semiestructurados (NoSQL). Se agregaron dos columnas de tipo JSON: espacio.datos_iot, que guarda las lecturas de los sensores de cada aula (escenario IoT), y publicacion.metadatos, que guarda etiquetas, adjuntos y geoubicación de las publicaciones (escenario big data).

Los entregables son: diccionario genérico actualizado, diccionario físico de cada SGBD, un script DDL por SGBD y un script de demostración de JSON por SGBD.

2. Justificación de la elección de los tres SGBD

Se eligieron PostgreSQL, MySQL y SQL Server por tres razones: son motores relacionales de uso real en empresas, cubren tres modelos distintos (código abierto con enfoque estándar, código abierto con enfoque de simplicidad y producto comercial integrado con el ecosistema Microsoft) y cada uno ofrece una forma diferente de almacenar JSON, que es la parte NoSQL del reto. Además, los tres se pueden instalar y probar en el mismo equipo de trabajo.

Tabla 1. Criterios de selección y valoración de los tres motores

Criterio	**PostgreSQL**	**MySQL**	**SQL Server**
Licencia	Libre y gratuita	Libre (GPL); edición comercial de Oracle	Comercial; la edición Express es gratuita con límite de tamaño por base
Soporte JSON	JSONB binario con índices GIN	Tipo JSON nativo con validación	Sin tipo nativo en esta versión: NVARCHAR + ISJSON y funciones JSON
Restricciones CHECK	Completas	Desde 8.0.16, con limitaciones sobre columnas de acciones referenciales	Completas
Acciones referenciales	Todas, sin limitaciones	Todas	Sin RESTRICT; limitación de rutas múltiples de cascada
Herramienta gráfica	pgAdmin 4	MySQL Workbench	SQL Server Management Studio
Lenguaje	SQL cercano al estándar (PL/pgSQL)	SQL con extensiones propias	T-SQL propietario

2.1 Por qué cada uno

- **PostgreSQL:** es el motor de referencia del curso y el más fiel al estándar SQL. Permite validar casi todas las reglas del modelo dentro de la base y tiene el

soporte JSON más completo (JSONB indexable). Es el punto de partida y el contra con el que se compara a los demás.

- **MySQL:** es el motor de código abierto más extendido en aplicaciones web. Se eligió para observar qué pasa con un modelo con muchas restricciones en un motor más simple y con reglas propias, y porque su tipo JSON nativo contrasta con el de PostgreSQL.
- **SQL Server:** es el motor corporativo más usado en el entorno Windows y .NET, y el que más se aprende en entornos empresariales. Se eligió porque su manejo de JSON es distinto (texto validado) y porque exige adaptar el modelo (sin RESTRICT, sin BOOLEAN, reglas de cascada propias).

2.2 Alternativas descartadas

Oracle Database se descartó por su licencia y su complejidad de instalación para un trabajo académico. SQLite no soporta la concurrencia de una red social, no respeta el tamaño de las columnas y no tiene un modelo de seguridad por usuario. Un motor NoSQL puro como MongoDB se descartó porque el modelo es profundamente relacional (34 claves foráneas) y el reto pide integrar un tipo NoSQL dentro de un motor relacional, no reemplazarlo.

3. Diccionario de datos genérico actualizado

El diccionario genérico describe la estructura lógica sin depender de un motor y refleja los diagramas ER y la normalización. Se mantiene el modelo de 25 tablas y 34 claves foráneas, normalizado hasta 3FN, con una actualización: dos columnas JSON opcionales. La actualización no rompe ninguna forma normal porque cada documento JSON depende únicamente de la clave de su fila y se trata como un valor atómico para el modelo relacional.

Tabla 2. Actualización del modelo respecto a la Tarea 3

Tabla	**Columna**	**Tipo genérico**	**Qué guarda**
publicacion	metadatos	JSON (opcional)	Etiquetas, adjuntos (tipo, url, tamaño) y geoubicación GeoJSON de la publicación.
espacio	datos_iot	JSON (opcional)	Identificador del sensor, fecha de la lectura, temperatura, humedad, CO2 y ocupación del aula.

El diccionario completo, con las 25 tablas, está en el archivo diccionario_generico.docx de la carpeta diccionarios.

4. Implementación paso a paso en cada SGBD

En los tres motores se siguió el mismo procedimiento: (1) preparar el entorno, (2) crear la base de datos y el esquema, (3) crear las tablas de las padres a las hijas, (4) crear los índices, (5) verificar y (6) probar los datos JSON. Los tres scripts se generaron a partir de una misma definición del modelo, por lo que tienen las mismas 25 tablas y 34 claves foráneas.

4.1 PostgreSQL 16 (SGBD 1)

- **Preparación:** PostgreSQL 16 con pgAdmin 4 como herramienta gráfica.
- **Base y esquema:** CREATE DATABASE red_social_pascualina y CREATE SCHEMA red_social. Todas las tablas viven en ese esquema.
- **Tablas:** claves primarias con GENERATED ALWAYS AS IDENTITY, CHECK para los valores fijos, UNIQUE simples y compuestas, y claves foráneas con ON DELETE y ON UPDATE explícitos.
- **JSON:** espacio.datos_iot y publicacion.metadatos son JSONB; un CHECK con jsonb_typeof exige que el valor sea un objeto. Se agregaron dos índices GIN para consultar dentro de los documentos.
- **Verificación:** el script se ejecutó sin errores y creó 25 tablas, 25 claves primarias, 34 claves foráneas, 13 restricciones UNIQUE, 22 CHECK y 24 índices (22 sobre claves foráneas y 2 GIN).

```
psql -U postgres -c "CREATE DATABASE red_social_pascualina;"
psql -U postgres -d red_social_pascualina -f ddl/scripts-ddl-postgresql/ddl_postgresql.sql
psql -U postgres -d red_social_pascualina -f ddl/scripts-ddl-postgresql/json_postgresql.sql
```

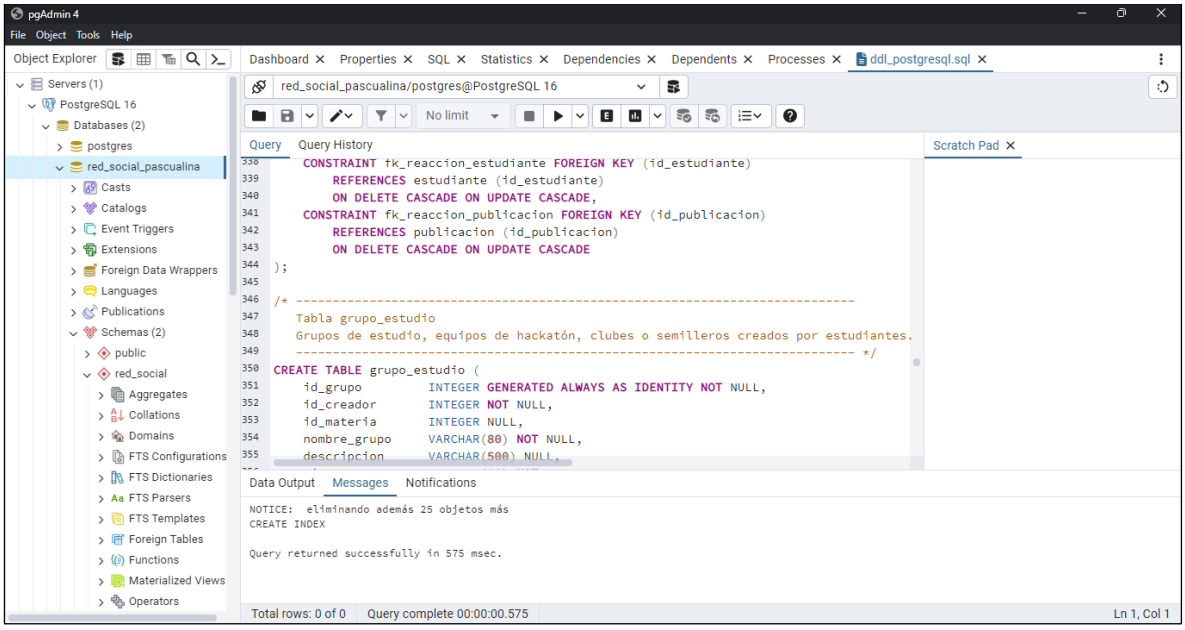


Figura 1. Ejecución del script ddl_postgresql.sql en pgAdmin 4: «Query returned successfully».

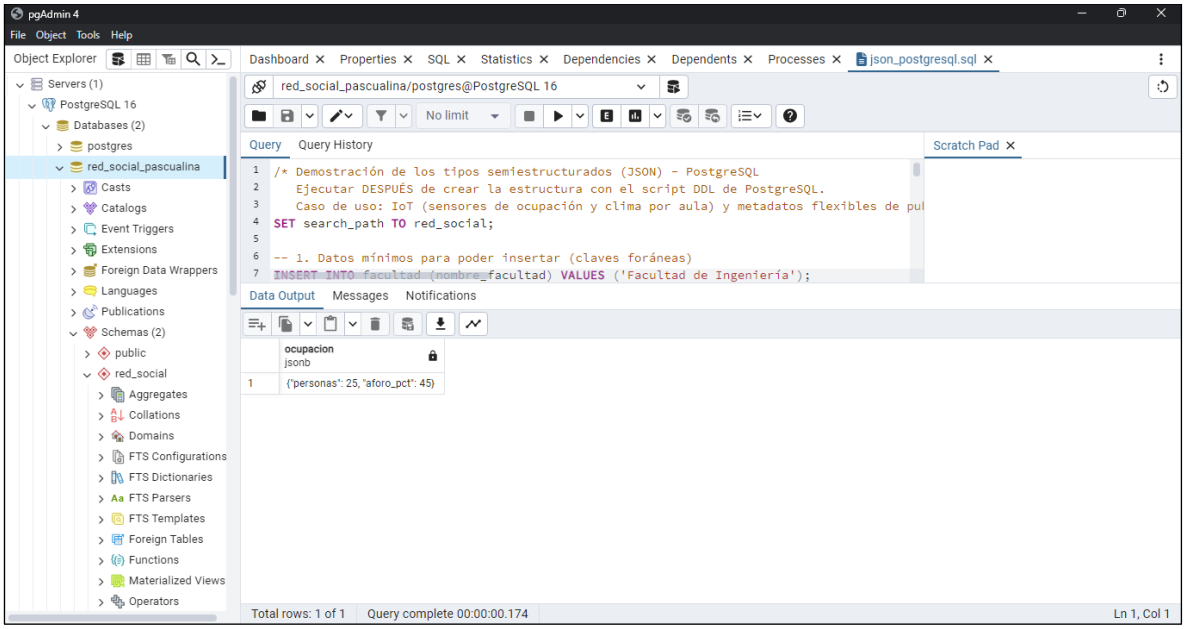


Figura 2. Resultado de json_postgresql.sql: la ocupación del aula quedó modificada en 25 personas mediante jsonb_set.

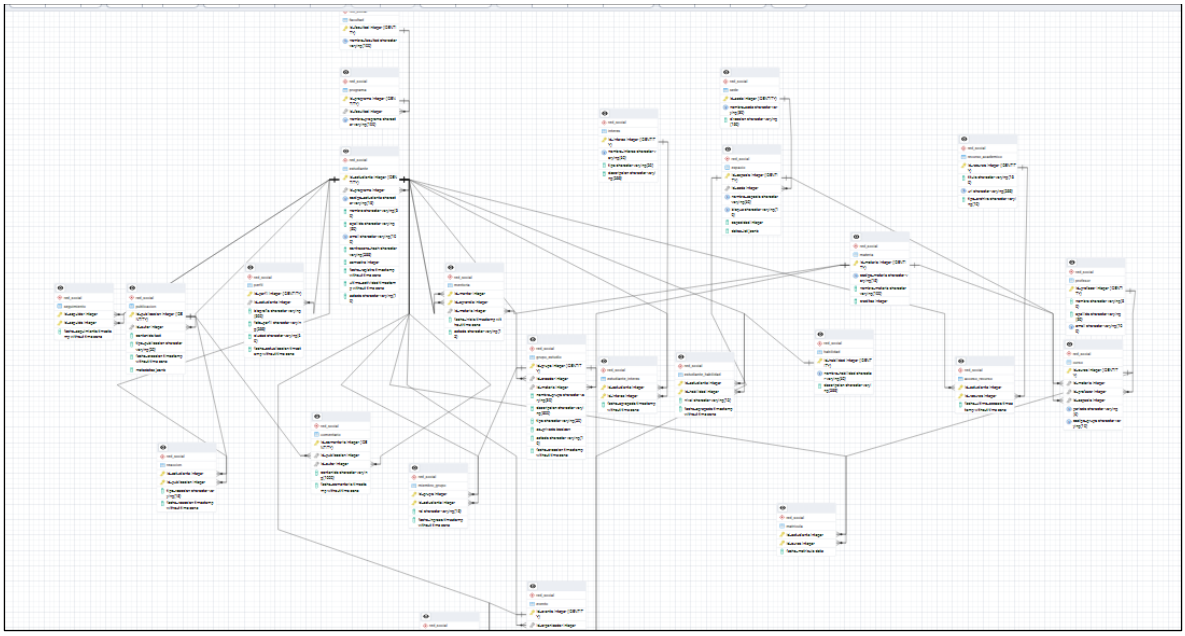


Figura 3. Diagrama entidad-relación del esquema red_social generado con pgAdmin 4 (ERD For Schema).

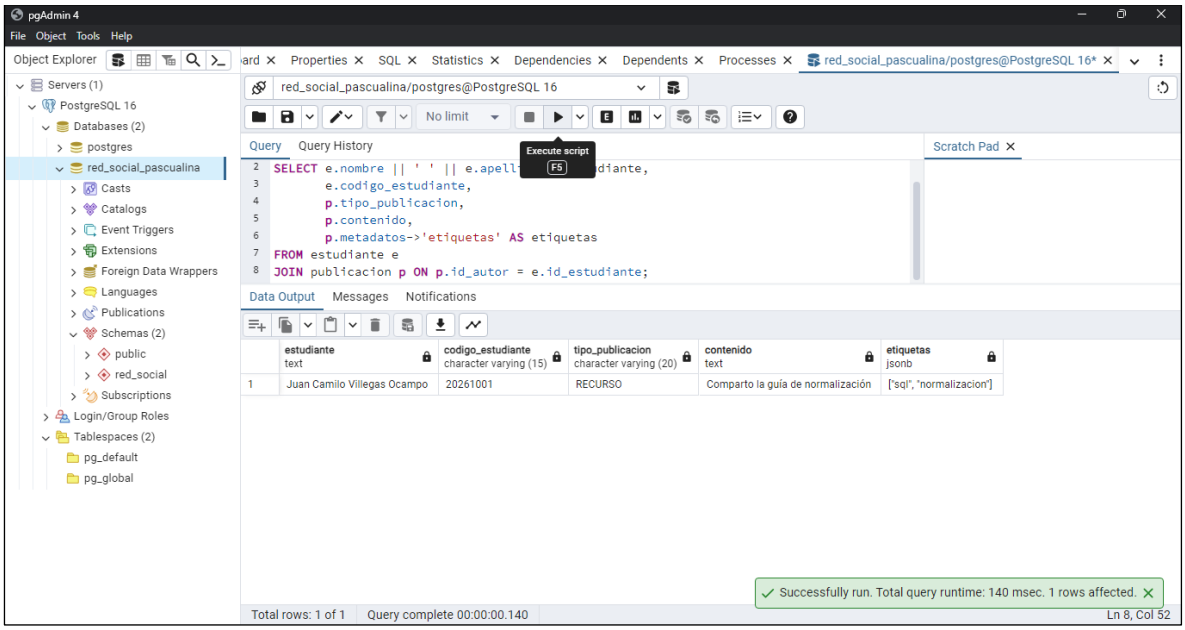


Figura 4. Consulta en pgAdmin 4 que une estudiante y publicacion mediante id_autor y extrae las etiquetas del campo JSONB.

4.2 MySQL 8 (SGBD 2)

- **Preparación:** MySQL 8 (el script requiere 8.0.16 o superior por los CHECK) y MySQL Workbench.

- **Base de datos:** en MySQL esquema y base de datos son lo mismo: el script crea red_social_pascualina con juego de caracteres utf8mb4.
- **Tablas:** INT con AUTO_INCREMENT, motor InnoDB (necesario para claves foráneas), BOOLEAN como alias de TINYINT(1), DATETIME en lugar de TIMESTAMP y DEFAULT (CURRENT_DATE) entre paréntesis para columnas DATE.
- **Ajuste propio del motor:** MySQL no permite un CHECK sobre una columna que tenga una acción referencial CASCADE o SET NULL (error 3823). Las reglas «un estudiante no se sigue a sí mismo» y «no es su propio mentor» se pasaron a triggers BEFORE INSERT y BEFORE UPDATE con SIGNAL, y fk_evento_espacio usa ON UPDATE RESTRICT.
- **JSON:** tipo JSON nativo, que valida el documento al insertar; un CHECK con JSON_TYPE exige que sea un objeto.

  ```
  mysql -u root -p < ddl/scripts-ddl-mysql/ddl_mysql.sql
  mysql -u root -p < ddl/scripts-ddl-mysql/json_mysql.sql
  ```

- **Verificación:** el script se ejecutó sin errores en MySQL 8.0.46 desde MySQL Workbench. La única advertencia fue la esperada del DROP DATABASE IF EXISTS en la primera ejecución (la base aún no existía). Al comienzo del trabajo el servidor local no iniciaba (error 1053 de Windows), por lo que se desinstaló y se reinstaló MySQL Server 8.0.46 con el instalador oficial.

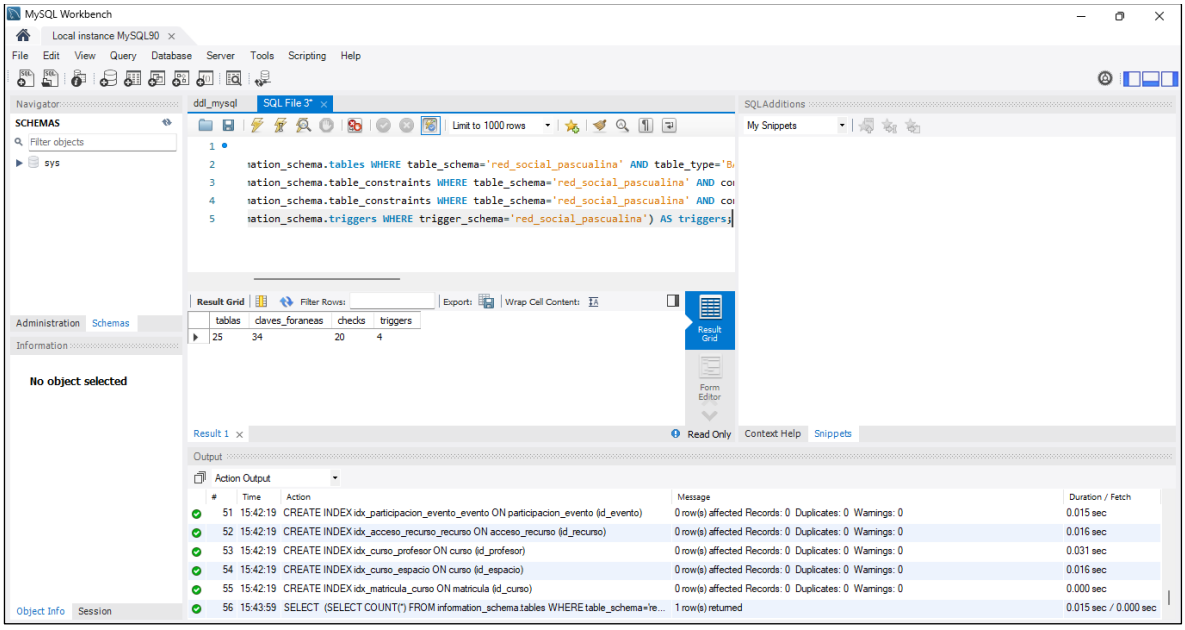


Figura 5. Verificación en MySQL Workbench: 25 tablas, 34 claves foráneas, 20 restricciones CHECK y 4 triggers.

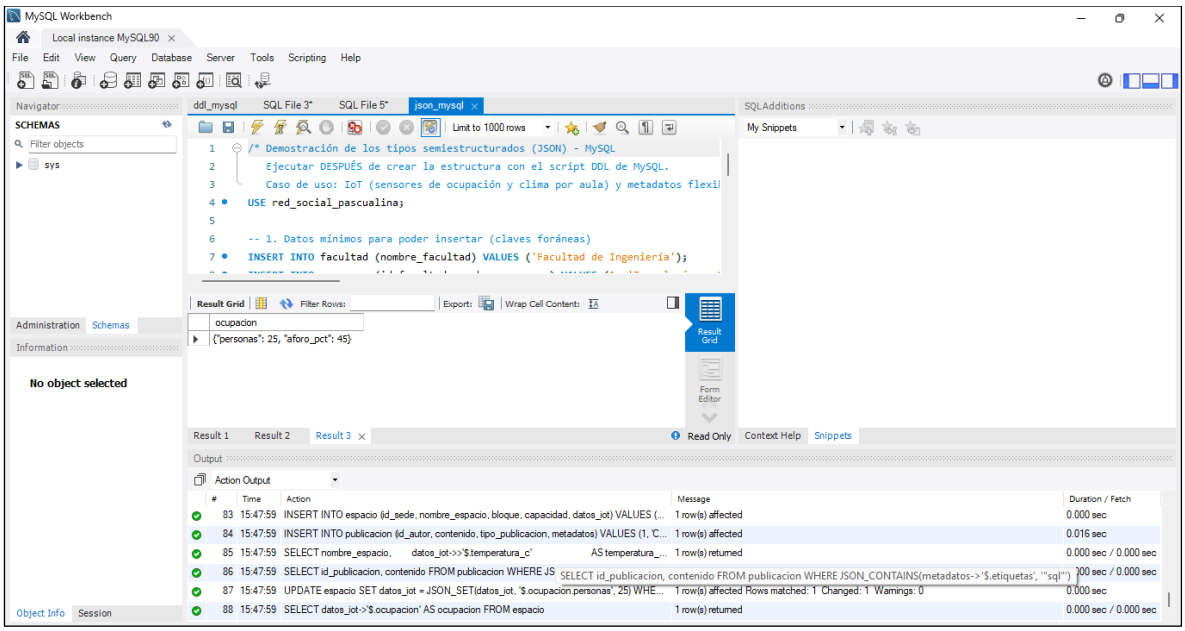


Figura 6. Resultado de json_mysql.sql en MySQL Workbench: consultas con JSON_CONTAINS y ->>, y ocupación modificada a 25 personas con JSON_SET.

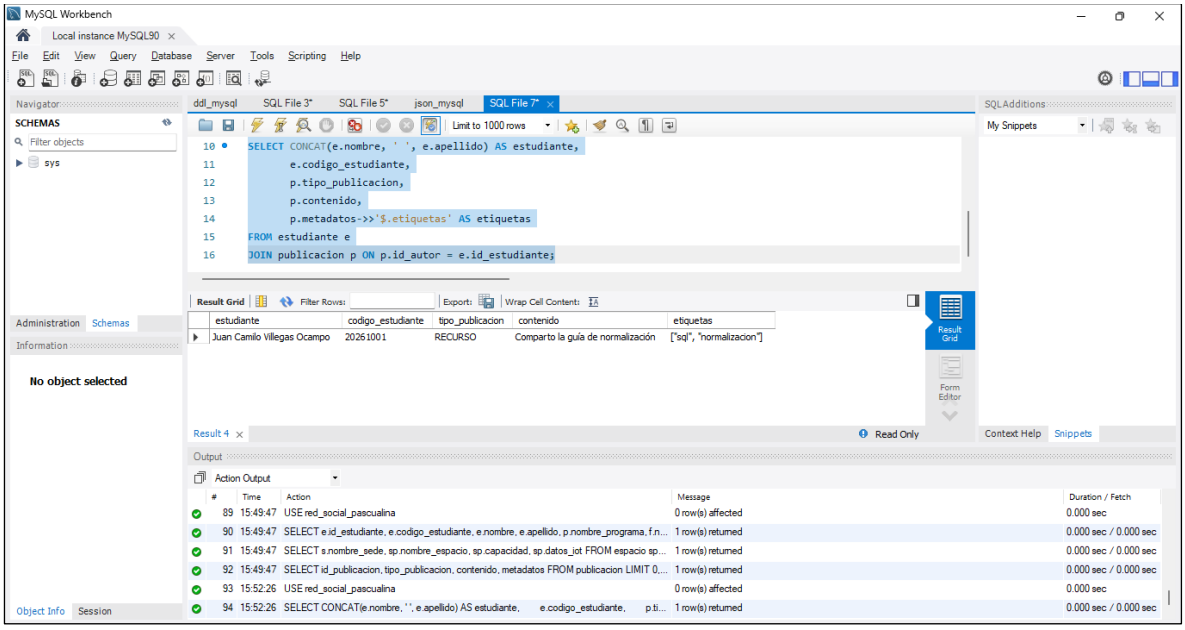


Figura 7. Consulta en MySQL Workbench que une estudiante y publicacion mediante la clave foránea id_autor y extrae las etiquetas del campo JSON.

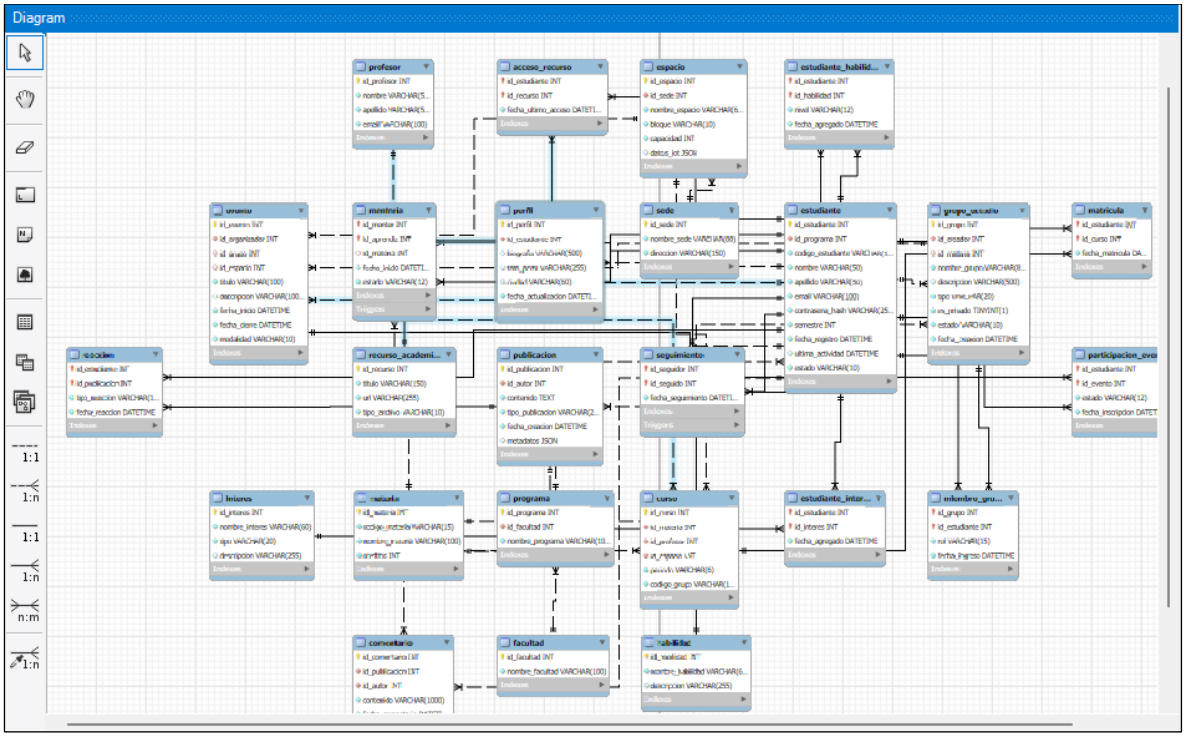


Figura 8. Diagrama EER del esquema red_social_pascualina generado con MySQL Workbench (Reverse Engineer): 25 tablas, sus claves foráneas y las columnas JSON datos_iot y metadatos.

4.3 Microsoft SQL Server 2022 (SGBD 3)

- **Preparación:** SQL Server 2022 Express y sqlcmd (también sirve SQL Server Management Studio).
- **Base y esquema:** CREATE DATABASE red_social_pascualina y CREATE SCHEMA red_social. El script puede ejecutarse varias veces porque borra las tablas existentes antes de crearlas.
- **Tablas:** INT con IDENTITY(1,1), NVARCHAR para guardar tildes y eñes, NVARCHAR(MAX) en lugar de TEXT (obsoleto), BIT en lugar de BOOLEAN y DATETIME2(0) para fecha y hora.
- **Ajuste propio del motor:** SQL Server no tiene RESTRICT, así que se usa NO ACTION. Al ejecutar el script apareció el error 1785 («puede causar ciclos o varias rutas de cascada»): una publicación y su autor llegan al mismo comentario por dos caminos de ON UPDATE CASCADE. Se resolvió usando ON UPDATE NO ACTION en las 34 claves foráneas (las columnas IDENTITY no se pueden modificar) y ON DELETE NO ACTION en seguimiento.id_seguido y mentoria.id_aprendiz.
- **JSON:** no hay tipo JSON nativo en esta versión: las columnas son NVARCHAR(MAX) con CHECK (ISJSON(col) = 1). Se consultan con JSON_VALUE, JSON_QUERY y OPENJSON, y se modifican con JSON_MODIFY.
- **Verificación:** el script se ejecutó sin errores en SQL Server 2022 Express: 25 tablas, 25 claves primarias, 34 claves foráneas, 13 UNIQUE, 22 CHECK y 22 índices.

```
sqlcmd -S .\SQLEXPRESS -E -i ddl\scripts-ddl-sqlserver\ddl_sqlserver.sql
sqlcmd -S .\SQLEXPRESS -E -i ddl\scripts-ddl-sqlserver\json_sqlserver.sql
```

Tabla 3a. Resultado de la verificación en SQL Server 2022 Express (consulta al catálogo con sqlcmd)

Objeto	Cantidad
Tablas del esquema red_social	25
Claves primarias	25
Claves foráneas	34
Restricciones UNIQUE	13

Objeto	Cantidad
Restricciones CHECK	22
Índices sobre claves foráneas	22

El script de demostración json_sqlserver.sql también se ejecutó sin errores: JSON_VALUE devolvió temperatura 24,5 y 18 personas, OPENJSON encontró la publicación con la etiqueta «sql» y JSON_MODIFY dejó la ocupación en 25 personas.

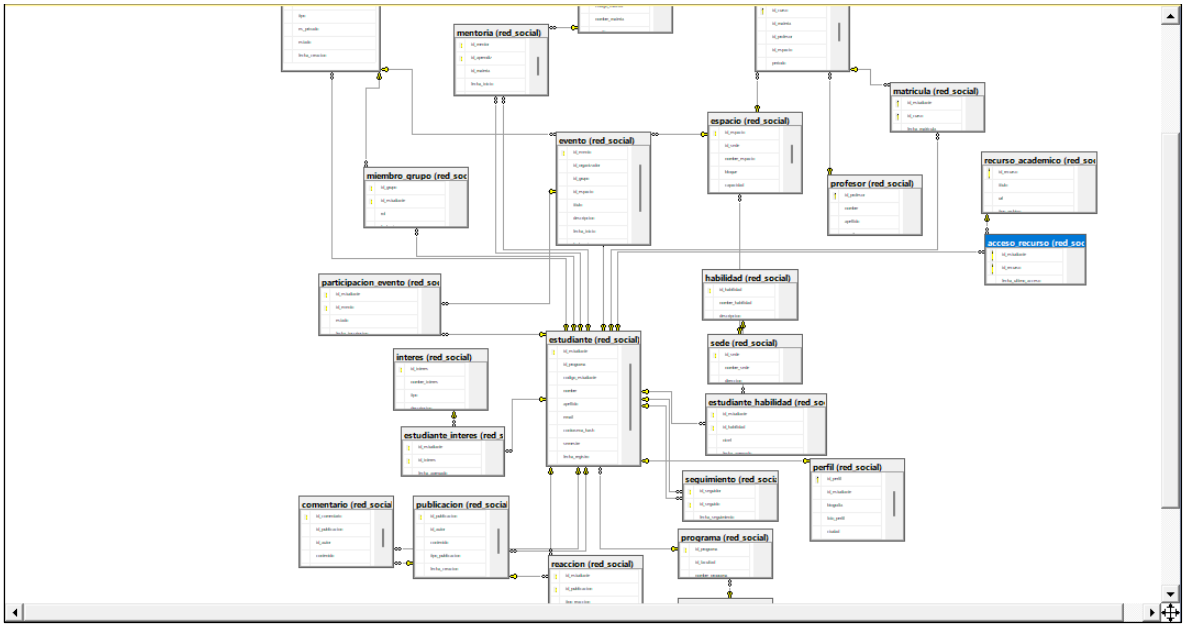


Figura 9. Diagrama de base de datos de red_social_pascualina en SQL Server Management Studio, con las tablas del esquema red_social y sus claves foráneas (vista parcial).

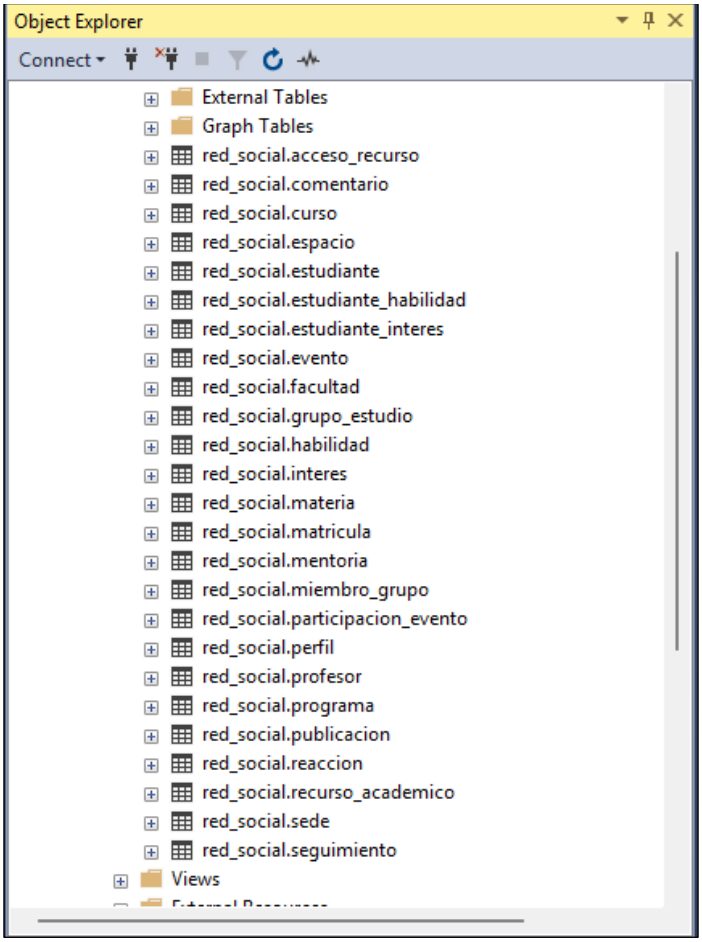


Figura 10. Las 25 tablas del esquema red_social en la base red_social_pascualina, vistas en SQL Server Management Studio.

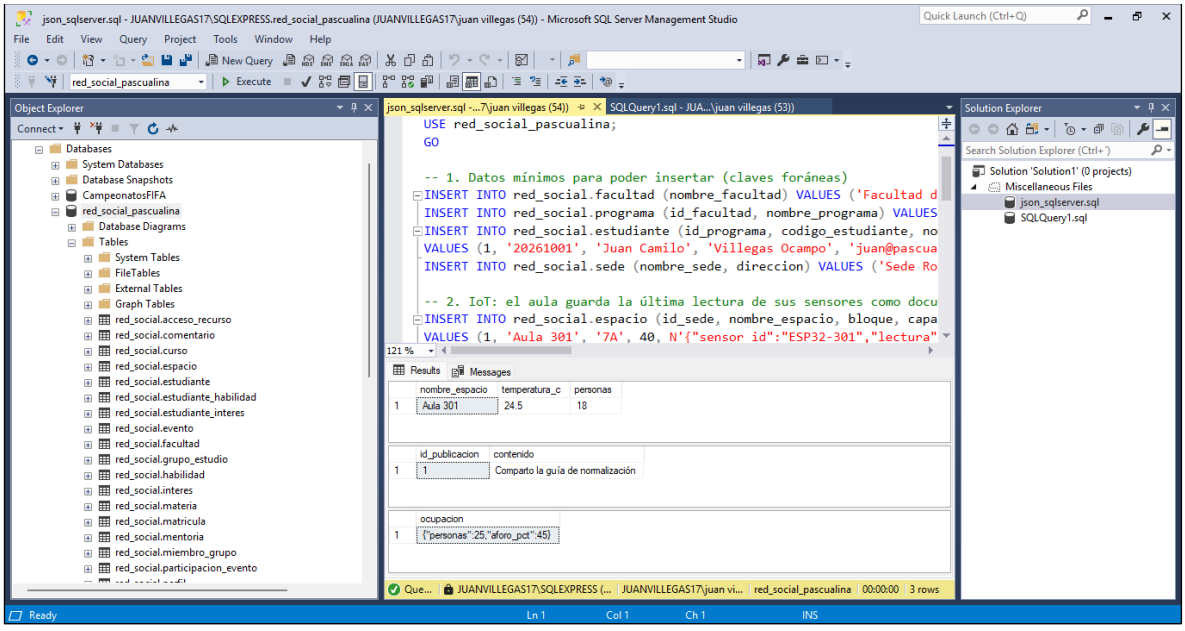


Figura 11. Resultado de json_sqlserver.sql en SQL Server Management Studio: lectura de valores con JSON_VALUE, búsqueda por etiqueta con OPENJSON y ocupación modificada con JSON_MODIFY.

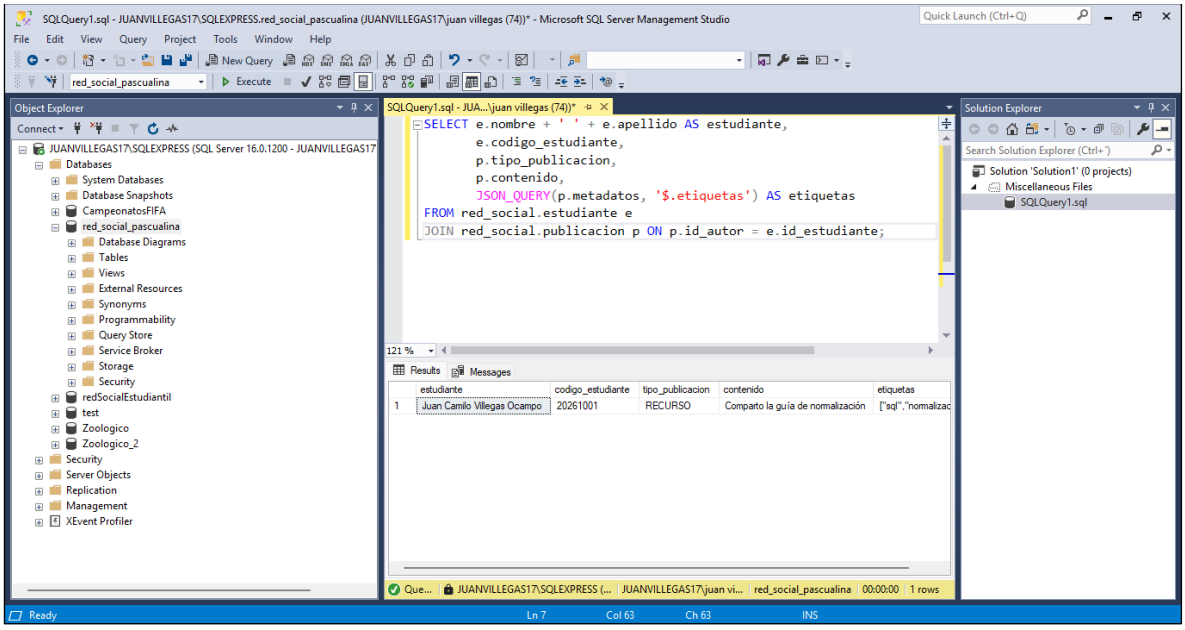


Figura 12. Consulta en SQL Server Management Studio que une estudiante y publicacion mediante id_autor y extrae las etiquetas con JSON_QUERY.

5. Comparación de los diccionarios físicos

Cada columna del modelo tiene el mismo significado en los tres motores; cambian el tipo físico, la sintaxis y algunas restricciones. Los diccionarios físicos completos están en los archivos diccionario_fisico_postgresql.docx, diccionario_fisico_mysql.docx y diccionario_fisico_sqlserver.docx.

Tabla 3. Equivalencia de tipos de datos entre los tres motores

Tipo genérico	**PostgreSQL**	**MySQL 8**	**SQL Server**
Entero autogenerado	INTEGER GENERATED ALWAYS AS IDENTITY	INT AUTO_INCREMENT	INT IDENTITY(1,1)
Texto corto	VARCHAR(n)	VARCHAR(n)	NVARCHAR(n)
Texto largo	TEXT	TEXT (hasta 64 KB)	NVARCHAR(MAX)
Booleano	BOOLEAN	BOOLEAN (TINYINT(1))	BIT
Fecha y hora	TIMESTAMP	DATETIME	DATETIME2(0)
Fecha	DATE	DATE	DATE
JSON	JSONB	JSON	NVARCHAR(MAX) + ISJSON
Valor por defecto fecha actual	CURRENT_TIMESTAMP	CURRENT_TIMESTAMP	SYSDATETIME()

Tabla 4. Equivalencia de reglas e integridad

Aspecto	**PostgreSQL**	**MySQL 8**	**SQL Server**
Esquema	Esquema red_social	Base de datos red_social_pascualina	Esquema red_social
ON DELETE RESTRICT	RESTRICT	RESTRICT	NO ACTION
ON UPDATE	CASCADE	CASCADE (RESTRICT en evento.id_espacio)	NO ACTION

Aspecto	PostgreSQL	MySQL 8	SQL Server
Regla «A distinto de B»	CHECK	Trigger BEFORE INSERT/UPDATE	CHECK
Formato de curso.periodo	CHECK con ~ (expresión regular)	CHECK con REGEXP	CHECK con LIKE
Validar JSON	jsonb_typeof(col) = 'object'	JSON_TYPE(col) = 'OBJECT'	ISJSON(col) = 1
Índice sobre JSON	GIN	Columna generada + índice	Columna calculada + índice
Cantidad de CHECK	22	20 (más 4 triggers)	22
Cantidad de índices	24	22	22

6. Tipos de datos SQL y NoSQL para big data e IoT

Los sistemas relacionales exigen que cada fila tenga las mismas columnas. En escenarios de big data e IoT eso es incómodo: los datos llegan de muchas fuentes, con campos distintos y que cambian con el tiempo. Por eso los tres motores incorporan almacenamiento semiestructurado en formato JSON dentro de una tabla relacional, lo que permite mantener las claves y relaciones del modelo y, a la vez, guardar documentos flexibles.

6.1 Caso de uso IoT: espacio.datos_iot

Cada aula puede tener sensores de ocupación, temperatura, humedad o CO2, con fabricantes y campos diferentes. Si cada medida fuera una columna, habría que modificar la tabla cada vez que se agregue un tipo de sensor. En cambio, una columna JSON guarda la última lectura tal como la envía el dispositivo, por ejemplo:

```
{"sensor_id":"ESP32-301","lectura":"2026-10-
20T10:15:00","temperatura_c":24.5,

"humedad_pct":58,"co2_ppm":612,"ocupacion":{"personas":18,"aforo_pct":45}
,"alertas":[]}
```

Esto responde preguntas útiles para la plataforma, como mostrar si un aula tiene cupo disponible antes de crear un evento presencial. La tabla espacio sigue teniendo su clave primaria, su sede y su capacidad; solo el dato variable es JSON.

6.2 Caso de uso big data: publicacion.metadatos

Las publicaciones de una red social cambian de forma: algunas llevan etiquetas, otras adjuntos, otras una ubicación. El campo JSON recibe esa variedad sin cambiar la estructura, y las etiquetas permiten búsquedas por tema:

```
{"etiquetas":["sql","normalizacion"],
 "adjuntos":[{"tipo":"PDF","url":"https://ejemplo.edu.co/guia.pdf","kb":820}],
 "geoubicacion":{"type":"Point","coordinates":[-75.5905,6.2796]}}
```

6.3 Justificación técnica de la elección del tipo

Tabla 5. Cómo guarda y consulta JSON cada motor

Aspecto	PostgreSQL	MySQL 8	SQL Server
Tipo	JSONB (binario descompuesto)	JSON (binario, validado al insertar)	NVARCHAR(MAX) con CHECK (ISJSON(col) = 1)
Leer un valor	datos_iot->>'temperatura_c'	datos_iot->>'$.temperatura_c'	JSON_VALUE(datos_iot, '$.temperatura_c')
Buscar en una lista	metadatos @> '{"etiquetas":["sql"]}'	JSON_CONTAINS(metadatos->'$.etiquetas', '"sql"')	OPENJSON(metadatos, '$.etiquetas')
Modificar una clave	jsonb_set(...)	JSON_SET(...)	JSON_MODIFY(...)
Indexar	Índice GIN directo	Columna generada + índice	Columna calculada + índice
Validación	Automática; CHECK exige objeto	Automática; CHECK exige objeto	Solo con el CHECK ISJSON

PostgreSQL y MySQL validan el documento por sí mismos; en SQL Server la validación depende del CHECK porque el dato es texto. JSONB se eligió en PostgreSQL y no JSON (texto) porque se consulta mucho más de lo que se escribe y el formato binario permite índices; JSONB descarta el orden y las claves duplicadas, algo irrelevante para lecturas de sensores. Para texto con acentos en SQL Server se usó NVARCHAR, no VARCHAR.

6.4 Optimización del tamaño del dato para señales de sensores

Una lectura en JSON ocupa decenas de bytes por texto y nombres de clave repetidos. Eso es aceptable para guardar solo la última lectura de cada aula. Si se quisiera conservar todo el historial de lecturas (miles de registros por aula por día), la mejora sería una tabla aparte lectura_sensor con una columna por medida y tipos pequeños (SMALLINT para CO2 y ocupación, REAL o DECIMAL(4,1) para temperatura, TIMESTAMP para la fecha) o un motor de series de tiempo; el JSON quedaría solo para los atributos variables del dispositivo.

6.5 Costos y riesgos del dato semiestructurado

- **Integridad:** Las claves dentro del JSON no tienen integridad referencial ni tipos fijos: un error de escritura (por ejemplo, temperatura como texto) no lo detecta la base.
- **Rendimiento:** Consultar dentro del JSON sin índice recorre toda la tabla; en PostgreSQL se resuelve con GIN y en los otros dos con columnas generadas o calculadas.
- **Criterio de uso:** Lo que se va a filtrar o a unir con frecuencia debe ser una columna normal, y el JSON debe reservarse para lo variable. Por eso la capacidad del aula y la sede siguen siendo columnas relacionales.

Este enfoque es coherente con los trabajos sobre comparación de modelos relacionales y NoSQL citados en la unidad (Fadhel y Jameel, 2022; Hasan et al., 2019; Herrera-Ramírez et al., 2021): los modelos híbridos permiten conservar la consistencia del relacional y aprovechar la flexibilidad del documento.

6.6 Prueba de funcionamiento

Cada motor tiene un script de demostración (json_postgresql.sql, json_mysql.sql, json_sqlserver.sql) que inserta un aula con su lectura IoT y una publicación con metadatos, y ejecuta consultas para leer valores, buscar por etiqueta y modificar una sola clave del documento. En PostgreSQL y SQL Server las consultas devolvieron lo esperado: temperatura 24,5 y 18 personas, la publicación con etiqueta «sql» y la ocupación actualizada a 25 personas. En PostgreSQL, además, un documento que no sea objeto (por ejemplo, una lista) fue rechazado por el CHECK.

7. Retroalimentación y ajustes

el trabajo se realizó de forma individual basado en las retroalimentaciones del docente y apoyo en los encuentros semanales recibieron observaciones sobre el desarrollo de la actividad. Por eso los ajustes que se describen a continuación provienen de las pruebas hechas con cada motor propias en base a prueba y error

Ajustes realizados durante el desarrollo, derivados de las pruebas con los motores:

- SQL Server rechazó el script por múltiples rutas de cascada (error 1785); se pasaron las claves foráneas a ON UPDATE NO ACTION y se dejó ON DELETE NO ACTION en las dos claves restantes de las relaciones reflexivas.
- Para MySQL se reemplazaron los CHECK sobre columnas con acciones referenciales por triggers, siguiendo la limitación documentada del motor (error 3823).
- Se agregaron las columnas JSON y su validación en cada motor, y los índices GIN en PostgreSQL.

8. Dificultades, ventajas y desventajas por SGBD

Tabla 6. Dificultades encontradas, ventajas y desventajas observadas

SGBD	Dificultades encontradas	Ventajas observadas	Desventajas observadas
PostgreSQL	Ninguna al ejecutar el script; hubo que decidir entre JSON y JSONB y definir los índices GIN.	Admite todo el modelo sin adaptaciones; JSONB indexable; DDL transaccional; mensajes de error con el nombre de cada restricción.	Administración y autenticación más extensas (roles y archivo pg_hba.conf), lo que puede complicar la primera instalación.
MySQL	Limitación documentada del error 3823 en CHECK; sintaxis distinta para DEFAULT de fecha; hay que usar InnoDB. La instalación local del servidor falló al inicio (error 1053) y hubo que reinstalarlo.	Instalación y uso sencillos; tipo JSON nativo con validación; sintaxis familiar.	Esquema y base de datos coinciden; BOOLEAN simulado; reglas «distinto de» solo con triggers; menos estricto con algunas restricciones.
SQL Server	Error 1785 por rutas de cascada; sin RESTRICT, BOOLEAN ni JSON nativo; TEXT obsoleto.	Buenas herramientas gráficas; integración con Windows y .NET; funciones JSON completas (OPENJSON, JSON_MODIFY).	Licencia comercial y límites de la edición Express; T-SQL propietario; el JSON no se valida solo.

9. Conclusiones y propuestas de mejora

9.1 Conclusión individual: Juan Camilo Villegas Ocampo

Con el desarrollo de esta actividad se concluye que un modelo lógico bien hecho no se implementa igual en todos los gestores. Cuando lo pasé de PostgreSQL a MySQL y a SQL Server, los tipos de datos y la sintaxis cambiaron, pero lo más importante fue entender que cada motor tiene sus propias reglas y limitaciones. Antes pensaba que bastaba con cambiar IDENTITY por AUTO_INCREMENT; ahora se sabe que también hay que revisar las restricciones y las acciones referenciales.

Lo que más me costó fueron los errores propios de cada motor. En SQL Server el script falló con el error 1785 por varias rutas de cascada, y tuve que entender por qué dos caminos de ON UPDATE CASCADE llegaban a la misma tabla antes de cambiarlos a NO ACTION. En MySQL, el error 3823 no permite un CHECK sobre una columna con acción referencial, así que dos reglas pasaron a ser triggers. También me tocó desinstalar y reinstalar MySQL, porque el servidor local no iniciaba (error 1053). Esa parte me enseñó que ejecutar el script en el motor real es la única forma de saber si funciona: la revisión de sintaxis no habría detectado el error 1175 del modo seguro de Workbench, que apareció al ejecutar el UPDATE sobre una columna que no era clave.

La parte de JSON me gustó porque conecta lo relacional con big data e IoT. Entendí que no se trata de reemplazar el modelo relacional, sino de dejar en columnas lo que se filtra y se une, como la capacidad del aula o el autor de la publicación, y guardar como documento lo que cambia de forma, como la lectura de un sensor. Además comprobé que cada motor lo trata distinto: PostgreSQL con JSONB indexable, MySQL con un tipo JSON validado y SQL Server con texto y la función ISJSON.

Como mejora futura, haría una tabla aparte para el historial de lecturas de los sensores, con tipos de dato pequeños, en lugar de guardar solo la última lectura. También cambiaría la clave de mentoria para permitir varias mentorías entre la misma pareja de estudiantes. Este trabajo me dejó más seguro para explicar por qué elegir un gestor u otro según lo que necesite el proyecto, y no solo por costumbre.

9.3 Propuestas de mejora

- Crear una tabla lectura_sensor para el historial de lecturas IoT, con tipos pequeños, y partirla por fecha.

- Cambiar la clave de mentoria para que admita varias mentorías por pareja de estudiantes (incluyendo id_materia).
- Agregar índices sobre campos frecuentes del JSON (por ejemplo, etiquetas de publicación) en MySQL y SQL Server mediante columnas generadas.
- Automatizar la ejecución y verificación de los tres scripts en un flujo de integración continua.

10. Referencias

Elmasri, R. y Navathe, S. A. (2007). Algoritmos de diseño de bases de datos relacionales y dependencias adicionales. En Fundamentos de sistemas de bases de datos (5.ª ed., pp. 317-344). Pearson Educación.

Fadhel, S. A. y Jameel, E. A. (2022). A comparison between nosql and rdbms: storage and retrieval. Minar International Journal of Applied Sciences and Technology, 4(3), 172-184. https://doi.org/10.47832/2717-8234.12.18

Hasan, M., Panidi, E. y Badenko, V. (2019). Comparative evaluation of nosql and relational databases performance while analyzing semi-structured geospatial data. International Scientific Conference GEOBALCANICA. https://doi.org/10.18509/gbp.2019.64

Herrera-Ramírez, J., Treviño-Villalobos, M. y Víquez-Acuña, L. (2021). Un motor de almacenamiento híbrido para datos geoespaciales utilizando paradigmas NoSQL y SQL. Revista Tecnología en Marcha, 34(1), 40-54. https://doi.org/10.18845/tm.v34i1.4822

Mannino, M. V. (2007). Administración de bases de datos: diseño y desarrollo de aplicaciones (3.ª ed.). McGraw-Hill Interamericana.

Microsoft Ignite. (2025, 12 de junio). ¿Qué es SQL Server Management Studio (SSMS)? https://learn.microsoft.com/es-es/ssms/sql-server-management-studio-ssms

Treviño-Villalobos, M., Víquez-Acuña, L., Quirós-Oviedo, R. y Víquez-Acuña, O. (2022). Evaluación del tiempo de respuesta de un geoservicio utilizando una base de datos híbrida y distribuida. Revista Colombiana de Computación, 23(1), 34-43. https://doi.org/10.29375/25392115.4228

PostgreSQL Global Development Group. (2024). PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/

Oracle Corporation. (2024). MySQL 8.0 reference manual: The JSON data type. https://dev.mysql.com/doc/refman/8.0/en/json.html

Microsoft. (2024). JSON data in SQL Server. https://learn.microsoft.com/en-us/sql/relational-databases/json/json-data-sql-server